

# Compiler-Guided Proactive KV Prefill for Agentic LLM Serving

## Abstract

Agentic LLM programs spend a large part of their wall-clock latency on prefill: every hop of a multi-agent workflow re-submits a long prompt whose prefix the serving engine may or may not still hold. Existing systems attack this by *predicting* which agent will be dispatched next, from an app-supplied step graph, an offline-trained predictor, or an online model of observed dispatches, and then moving or warming the corresponding key-value (KV) cache. We observe that for programs written against a language-integrated LLM abstraction, the prediction problem is largely unnecessary: because the prompt is assembled by the compiler rather than by the application, a static analysis can recover the call topology, the provenance of every argument, and the split between the invariant and variant parts of each prompt, with zero annotation, zero training, and zero observation of a running workload.

We present ProPrefill, a compiler-runtime co-design that turns this compile-time knowledge into idle-time work. The compiler emits a may-happen call topology together with binding provenance — which value produced now will feed which parameter later — and rewrites the prompt layout so that reusable content is contiguous. The runtime uses that information to speculatively prefill KV during engine idle windows, in particular the gaps opened by tool calls and retrieval, so that a subsequent call arrives at an already-populated prefix. Speculation is advisory: the prompt actually served is the runtime’s own rendering, so a wrong guess costs a cache miss and never a wrong prompt.

We report two verification case studies on Qwen3-4B on a single RTX 3090. On a multi-intent workflow with a supervisor and four subagents, ProPrefill reduces summed per-request time-to-first-token ( $\Sigma\text{TFT}$ ) from 469.9 ms to 265.1 ms ( $1.77\times$ ) against a vLLM automatic-prefix-caching baseline, capturing 72% of the headroom to a fully-cached Oracle at 186.2 ms, with per-component gains up to  $5.07\times$ . On a single-agent ReAct loop with a data-dependent back-edge, the terminal call site improves from 66.8 ms to 39.2 ms ( $1.71\times$ ). These are early, single-configuration results. ProPrefill is complementary to systems that govern KV *residency*: it governs KV *creation*.

## 1 Introduction

LLM applications are increasingly written as programs rather than as single prompts. A request is decomposed into a sequence of specialized agents, each of which submits its own prompt to a shared serving engine, and each of those prompts repeats a large body of content — system instructions, tool definitions, few-shot examples, accumulated context — that a previous call already paid to encode. Modern engines exploit this with prefix caching [6, 15], but a cache only helps if the right entry is resident at the right moment, and agentic traffic is adversarial to recency-based replacement: the natural reuse distance of an agent’s context spans the entire rest of the workflow.

A line of recent systems responds by trying to know the future. KVFlow abstracts the execution schedule as an agent step graph and prefetches the tensors an agent will need in the next step [7]. PBKV predicts agent invocations several steps ahead by fusing historical workflows with the context of the target workflow [14]. CacheScout learns agent execution transitions online, with no predefined workflow graph and no offline training, and issues a lightweight synthetic warmup request to materialize the predicted agent’s reusable anchor [13]. Pythia goes furthest: it mines workflow structure from logged execution traces and prefills not only shared prefixes but argument content recorded in earlier executions [12]. All four place the same bet: that what the program will do next can be reconstructed from outside the program. Helium moves the knowledge into a compilation step instead — and obtains it by having the developer write the workflow down as a query plan in a dedicated DSL [8].

**Observation.** The bet is only necessary because the serving layer is downstream of the program, and the DSL is only necessary because the host language is assumed to be opaque. In a language-integrated LLM abstraction — we build on byLLM, the runtime of the meaning-typed programming model implemented in the Jac language [3] — the prompt is not handed to the runtime by the application; it is *assembled* by the compiler and runtime from typed program constructs. A significant portion of what will eventually be sent to the engine is therefore fixed at compile time. What is not fixed is small and precisely characterized: the values that will be bound to the parameters of each call. This changes the ques-

tion from *which agent runs next?* to *which prefixes are already knowable*, and *when does each argument become available?* — and both are answerable by static analysis.

**Approach.** We build ProPrefill, a compiler–runtime co-design. The compiler recovers a may-happen call topology as a BFS-layered reachable set, recovers *binding provenance* linking each produced value to the future parameters it will feed, and separates the invariant from the variant part of every prompt. Because it owns prompt assembly, it also rewrites the layout: reusable zones are ordered first, and values that arrive late are confined to a trailing binding zone, so that the reusable prefix is a genuine prefix rather than being fragmented by interleaved dynamic content. The runtime consumes this to prefill KV during engine idle windows — both the temporal idle opened by a tool call or a retrieval round trip and the spatial idle available under chunked prefill [1] — constructing the longest knowable prefix: invariants, graph context, already-bound argument values, and the grammar compilation needed for constrained decoding [4]. Because the served prompt remains byLLM’s own rendering, speculation is advisory: a wrong guess costs a cache miss, never a semantic change.

### Contributions.

1. A compiler-based static analysis that recovers, for byLLM programs and with zero annotation and zero training, the call topology, the provenance of each parameter, and the invariant portion of each runtime prompt, and that automatically lays out every LLM prompt for maximal KV reuse — content that byLLM’s stock runtime renders in a fragmented order.
2. A runtime that proactively and speculatively prefills KV cache in engine idle windows using that compile-time structure, reducing cold-start TTFT.
3. An evaluation of the resulting compiler–runtime co-design on two structurally different agent topologies; a broader study across agent applications is ongoing.

**Positioning.** We are not competing with KV residency management. KVFlow and PBKV govern which KV *stays* on the GPU; ProPrefill governs which KV *comes into existence*, and when. The two compose: a residency policy has more to work with when the entries it is ranking were created ahead of demand rather than on the critical path.

## 2 Motivation

### 2.1 Who cares about TTFT

Before arguing that prefill can be moved off the critical path, it is worth stating why the quantity being moved

matters.

**TTFT is a provisioning metric.** Serving systems are sized and evaluated against TTFT service-level objectives. Cutting TTFT at fixed hardware means more requests are served inside the same SLO, which is a capacity statement, not only a user-experience one.

**Prefill is the part of latency still addressable at the systems layer.** Decode time is set by memory bandwidth and by how many tokens the model chooses to emit. A serving system cannot shorten it without changing the model or the output. Prefill, by contrast, is work whose *timing* the system controls.

**Prefill removed from the critical path is throughput, not just latency.** Prefill and decode contend for the same GPU. Work performed during an idle window is work the loaded engine no longer has to interleave with decode. From the serving system’s point of view the underlying quantity above TTFT is goodput, and relocating prefill improves it in a way that trimming a latency constant does not.

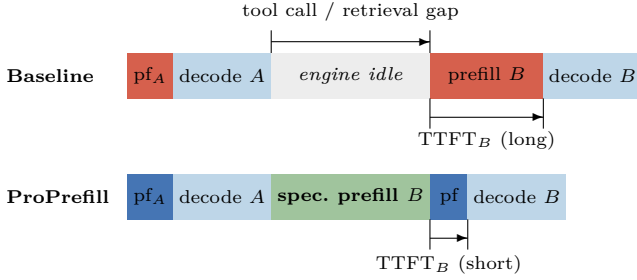
**Agentic call sites are prefill-heavy by construction.** In many agent applications the individual LLM calls consume a large context and emit a small structured output. For such call sites TTFT is a dominant share of the call’s latency, so it is the term worth attacking.

Concretely, three parties benefit: the application developer, through perceived responsiveness; the operator, through SLO attainment per GPU; and single-GPU or local deployments, where there is no spare replica to hide a cold prefill behind.

### 2.2 Why a compiler can know the prompt

Figure 1 shows the structural opportunity. An agent program alternates between LLM calls and non-LLM work — tool invocations, retrieval, environment interaction. During the non-LLM work the engine has nothing to do for this request. That gap is not incidental; it is the defining rhythm of agentic execution, and it recurs at every hop.

The question is what can usefully be computed inside the gap. Filling it requires knowing the content of a prompt that has not been issued yet. For a program written against byLLM, most of that content is already determined: the meaning-typed abstraction constructs the prompt from typed program constructs rather than from developer-authored strings [3], so the instruction text, the type and schema context, and the surrounding graph context are all fixed once the program is compiled. What is released only at runtime is the set of parameter values — and even those are not uniformly late: many of



**Figure 1:** The idle window created by a tool call or retrieval round trip. In the baseline the engine is idle through the gap and pays a cold prefill for the next call. ProPrefill spends the gap constructing the longest knowable prefix of that call, so the demand-time prefill shrinks to the residual.

them are computed by an earlier statement, well before the call that consumes them is reached. A static analysis that tracks where each argument is produced can therefore tell the runtime not only *what* the future prompt will look like but *how much of it is already available now*.

## 3 Design

ProPrefill has two halves: a compile-time analysis that produces structure, and a runtime that spends idle GPU time against that structure.

### 3.1 Static analysis over byLLM programs

The analysis runs as a compiler pass and requires no annotation from the developer and no trained model. It produces three artifacts.

**Call topology.** The pass deduces the topology of byLLM calls: which call sites can reach which other call sites, expanded as a BFS-layered reachable set rather than a single next-step guess. The result is a may-happen relation, so a branch whose outcome is not statically decidable contributes all of its successors rather than none of them.

**Argument provenance.** For every parameter of every call site, the pass records where the value that will be bound to it is produced. This is the artifact that distinguishes ProPrefill from next-agent prediction: knowing that call site  $v$  will run is far less actionable than knowing that the value being computed right now is the third argument of  $v$ , and is therefore prefillable right now.

**Invariant/variant split.** Each runtime prompt is partitioned into the portion that is fixed at compile time and the portion that depends on runtime values. The invariant portion is the part that can be materialized without any speculation at all.

### 3.2 Cache-aware prompt layout

Because the compiler owns prompt assembly, it does not have to accept the prompt as given. byLLM’s stock rendering interleaves invariant and variant content, which fragments what would otherwise be a long shared prefix: a single early-positioned dynamic value invalidates every token after it for reuse purposes. The pass rewrites the layout so that zones are ordered by expected stability — invariant instruction and schema content first, graph context next, and a trailing binding zone whose internal order follows the arrival order recovered by the provenance analysis. The effect is that the prefix that is reusable is also contiguous, which is what a prefix cache is able to exploit.

This is an authority a serving-layer system structurally does not have. A system that sits between the agent framework and the engine [12, 13] can decide what to keep, evict, or warm, but it receives the prompt already rendered.

### 3.3 Runtime: prefilling in engine idle windows

The runtime consumes the compiler’s artifacts and issues prefill work when the engine would otherwise be underutilized. It targets two kinds of idle. *Temporal* idle is the wall-clock gap in which no work is outstanding for the request, opened by a tool call or a retrieval round trip. *Spatial* idle is capacity within a scheduling step that is not consumed by the current batch; ProPrefill reaches it through chunked prefill [1], which lets prefill tokens be admitted incrementally alongside ongoing work rather than as an indivisible unit.

Within an idle window, the runtime builds the longest knowable prefix of a reachable future call: its invariant zone, the graph context that is already determined, the argument values that provenance says are already bound, and the grammar compilation required for constrained decoding of the call’s structured output [4]. This is a strictly larger unit than a fixed anchor. Where a warmup-based approach materializes the recurring context of a predicted agent, ProPrefill additionally materializes the speculated argument content, which is the part of the prompt that content-level approaches set aside.

### 3.4 Proactive binding of early-computed arguments

The single most productive case falls directly out of provenance: an argument that is prepared *before* the byLLM function that consumes it is called. Such a value is not speculative at all — it exists, it is final, and its consumer is statically known. The runtime prefills it as soon as it is produced, so by the time control reaches the consuming call site the corresponding KV is already resident. Section 4 shows that this case accounts for

the bulk of the measured gain: call sites whose prompts accumulate the outputs of earlier stages are precisely the ones that improve most.

### 3.5 Resolving supervisor branches

A supervisor that selects one of several subagents creates a branch that the static topology can only over-approximate. ProPrefill includes a training-free mechanism to narrow it: truncate the supervisor’s reasoning and fork a probe decode to see which successor it is heading toward, or equivalently run a draft model to speculate the routing decision, and prefill that successor first. The mechanism requires no additional training and no workload-specific tuning.

We note explicitly that this component is *not enabled* in the measurements reported in Section 4: the configuration evaluated there uses compiler-derived speculative prefill only, with no probe and no deploy-time prewarm. The consequence is visible in the results, where the call sites immediately downstream of the routing decision show no improvement.

### 3.6 Correctness model

Speculation in ProPrefill is advisory and byte-verified. The prompt that is served is byLLM’s own rendering of the call, unchanged; the speculatively constructed KV is used only if the tokens it covers match that rendering exactly. A mispredicted branch, a stale value, or a layout mismatch therefore degrades to a cache miss and a normal cold prefill. There is no configuration in which speculation changes the prompt the model sees, and hence none in which it changes the program’s output.

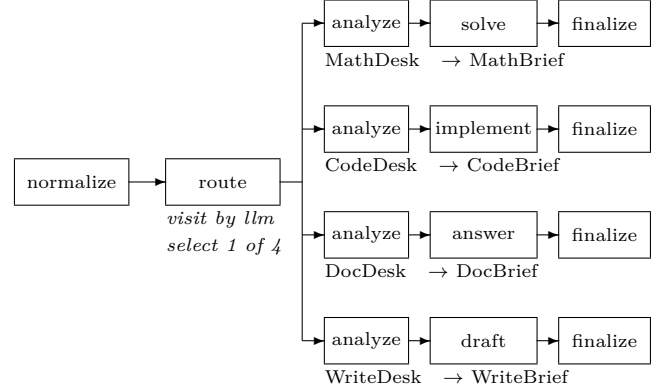
## 4 Evaluation

The evaluation reported here consists of two verification case studies. Its purpose is to establish that the compile-time structure ProPrefill recovers is sufficient to move real prefill work off the critical path, and to locate exactly which call sites benefit and which do not. It is not a general performance study; Section 5 states the scope precisely.

### 4.1 Setup

Both case studies run Qwen3-4B [10] on a single NVIDIA RTX 3090. ProPrefill is built on vLLM [6], and the baseline is therefore the same byLLM runtime served by vLLM with its automatic prefix caching (APC) enabled. We compare three configurations:

- **Baseline:** the same byLLM runtime with vLLM’s automatic prefix caching.
- **Oracle:** everything cached, no prefill required. Oracle is *measured*, not modelled: an identically-sized



**Figure 2:** Case study 1: a supervisor routes to one of four subagents, each a three-stage pipeline separated by retrieval.

prompt is replayed twice and the second serve is read, yielding a 100% cache hit.

- **ProPrefill:** compiler-derived speculative prefill, with no probe decode and no deploy-time prewarm.

Prefill-decode aggregation is disabled in these runs because the 3090 cannot sustain it for this model. All reported values are median TTFT: for case study 1, over 20 cold-start requests; for case study 2, over the calls that 20 multi-hop questions issue at each call site. The measurements are first requests, not steady-state requests behind a warm cache.

### 4.2 Case study 1: multi-intent supervisor with four subagents

The first topology, shown in Figure 2, is a multi-intent agent with a supervisor dominating four subagents. An entry stage normalizes the request, a routing stage selects one of four candidate desks through an LLM-driven graph visit, and the selected desk runs a three-stage pipeline whose middle stages are separated by retrieval. Each desk terminates in a typed brief. The retrieval steps and the tool calls between stages are what open the idle windows ProPrefill spends. We run 20 problems sampled from public datasets including GSM8K [2].

Table 1 reports per-component TTFT, and Figure 3 shows the same data graphically. The pattern is sharp and it is the pattern the design predicts.

**Where there is no gain, and why.** The entry stage (*normalize*) is already at the floor: at 193 prompt tokens its baseline TTFT of 37.3ms is 2.0ms above the 35.3ms Oracle, so there is essentially nothing to remove. The routing stage and all four *analyze* stages show 1.00× and capture 0% of their headroom, even though their Oracle values show that headroom exists — *route*, for instance, sits at 65.8ms against a 37.8ms Oracle. This is the expected consequence of running without the probe mechanism of Section 3: until the supervisor has emitted

**Table 1:** Case study 1: per-component median TTFT over 20 cold-start requests, Qwen3-4B on one RTX 3090. Oracle is measured by replaying one identically-sized prompt twice and reading the second serve. “Headroom captured” is the fraction of the baseline-to-Oracle gap that ProPrefill closes.

| component            | $n$ | prompt tok | baseline        | ProPrefill      | Oracle          | vs. baseline | headroom capt. |
|----------------------|-----|------------|-----------------|-----------------|-----------------|--------------|----------------|
| normalize (entry)    | 20  | 193        | 37.3 ms         | <b>37.6 ms</b>  | 35.3 ms         | 0.99×        | n/a (at floor) |
| route (visit-by-llm) | 20  | 549        | 65.8 ms         | <b>65.9 ms</b>  | 37.8 ms         | 1.00×        | 0%             |
| MathDesk.analyze     | 8   | 500        | 65.8 ms         | <b>66.1 ms</b>  | 37.8 ms         | 1.00×        | 0%             |
| CodeDesk.analyze     | 4   | 423        | 64.0 ms         | <b>64.3 ms</b>  | 34.5 ms         | 1.00×        | 0%             |
| DocDesk.analyze      | 3   | 283        | 52.3 ms         | <b>52.3 ms</b>  | 35.2 ms         | 1.00×        | 0%             |
| WriteDesk.analyze    | 5   | 294        | 52.3 ms         | <b>52.5 ms</b>  | 36.6 ms         | 1.00×        | 0%             |
| MathDesk.solve       | 8   | 879        | 119.8 ms        | <b>67.8 ms</b>  | 37.7 ms         | 1.77×        | 63%            |
| CodeDesk.implement   | 4   | 835        | 117.9 ms        | <b>49.7 ms</b>  | 36.2 ms         | 2.37×        | 83%            |
| DocDesk.answer       | 3   | 707        | 102.3 ms        | <b>62.3 ms</b>  | 37.1 ms         | 1.64×        | 61%            |
| WriteDesk.draft      | 5   | 676        | 102.0 ms        | <b>50.7 ms</b>  | 37.9 ms         | 2.01×        | 80%            |
| MathDesk.finalize    | 8   | 1453       | 216.3 ms        | <b>42.7 ms</b>  | 40.1 ms         | 5.07×        | 99%            |
| CodeDesk.finalize    | 4   | 1292       | 187.0 ms        | <b>42.6 ms</b>  | 37.2 ms         | 4.39×        | 96%            |
| DocDesk.finalize     | 3   | 1296       | 185.0 ms        | <b>41.5 ms</b>  | 39.2 ms         | 4.46×        | 98%            |
| WriteDesk.finalize   | 5   | 1258       | 171.1 ms        | <b>41.9 ms</b>  | 39.6 ms         | 4.09×        | 98%            |
| sum per request      | 20  | ~3300      | <b>469.9 ms</b> | <b>265.1 ms</b> | <b>186.2 ms</b> | <b>1.77×</b> | <b>72%</b>     |

its routing decision, the four desk-entry prompts are equally reachable and none of them is worth committing the idle window to.

**Where the gain is, and why.** The middle stages (solve, implement, answer, draft) improve by 1.64–2.37× and capture 61–83% of their headroom. Their prompts contain the output of the preceding analyze stage, which provenance identifies as an argument of a statically known consumer, and the retrieval step between the two opens the window in which to prefill it. The terminal stages improve most: 4.09–5.07×, capturing 96–99% of headroom, landing within 2.3–5.4 ms of their respective Oracle values. These are the longest prompts in the workflow (1258–1453 tokens) because they accumulate everything the desk has produced, and almost all of that content is available before the call is issued.

**Cost where there is no benefit.** On the six components that do not improve, ProPrefill is never more than 0.3 ms slower than the baseline (37.3→37.6, 65.8→65.9, 65.8→66.1, 64.0→64.3, 52.3→52.3, 52.3→52.5). Speculative prefill that does not pay off does not visibly cost, at least at this scale.

**Aggregate.** Summed over the components a single request executes,  $\Sigma$ TTFT falls from 469.9 ms to 265.1 ms, a 1.77× reduction, against a measured Oracle of 186.2 ms. ProPrefill therefore captures 72% of the total headroom that any KV-reuse system could obtain on this hardware and workload.

### 4.3 Case study 2: a ReAct tool loop

The second topology tests a structure the first does not contain: a data-dependent loop. It is a single-agent ReAct-style loop [11], shown in Figure 4, that alternates

**Table 2:** Case study 2: median TTFT per call site, Qwen3-4B on one RTX 3090, 20 multi-hop questions. The loop is data-dependent, so the two configurations do not execute the same number of calls; call counts are given for both.

| call site | calls (b/o)    | base           | ProPrefill     | ratio        | cached tok       |
|-----------|----------------|----------------|----------------|--------------|------------------|
| reason    | 74 / 84        | 39.3 ms        | 39.3 ms        | 1.00×        | 448 → 448        |
| integrate | 58 / 71        | 39.4 ms        | 38.3 ms        | 1.03×        | 368 → 544        |
| finalize  | <b>20 / 20</b> | <b>66.8 ms</b> | <b>39.2 ms</b> | <b>1.71×</b> | <b>304 → 496</b> |

reasoning with retrieval for two to five turns before emitting a typed brief.

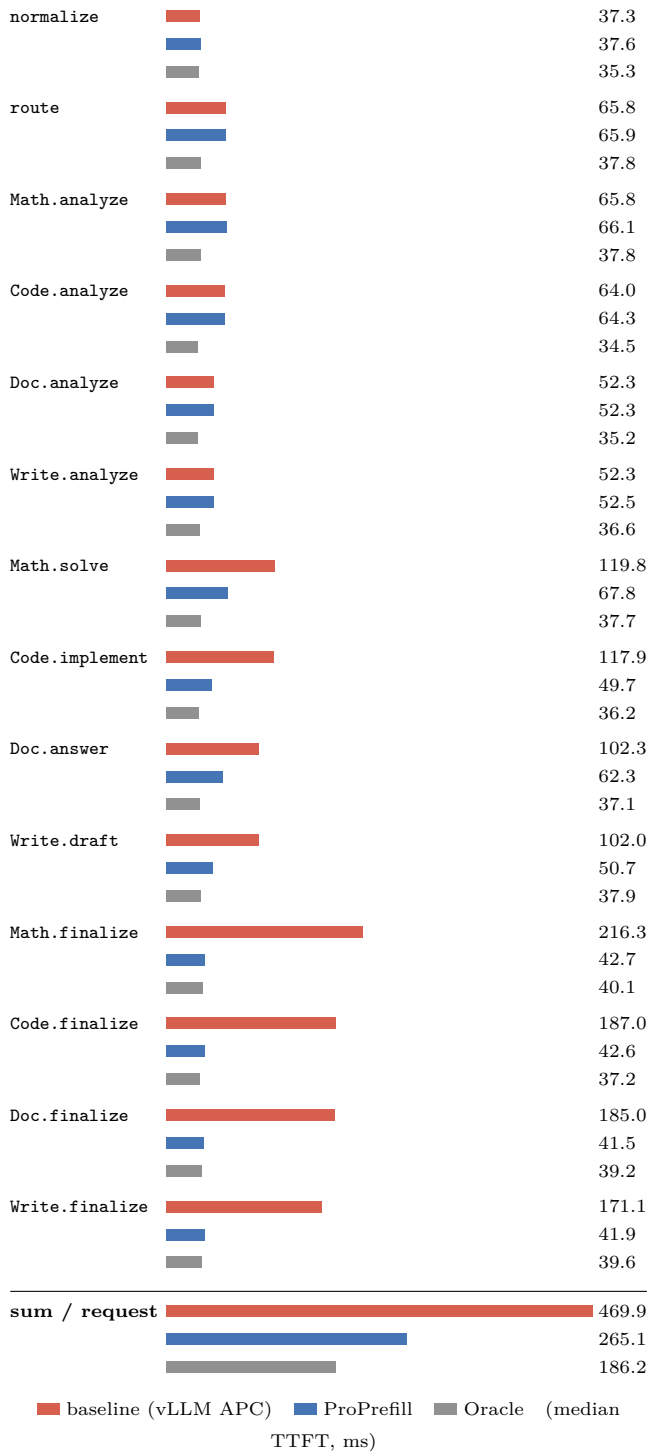
The compiler recovers the loop back-edge — **integrate** may be followed by **reason**, **integrate**, or **finalize** — and, more usefully, a self-referential return across it: the running notes are the return value of **integrate** and simultaneously an argument of the next **reason**, of the next **integrate**, and of **finalize**. That is a provenance fact, not a topology fact, and it is what makes the loop prefillable at all.

We run 20 multi-hop questions over a closed synthetic corpus of 37 passages constructed so that no passage contains both ends of a hop, forcing each hop to issue its own retrieval.

Table 2 reports the result, and Figure 5 plots it. Three observations follow.

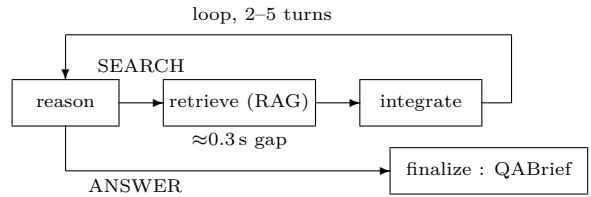
First, **reason** shows no improvement (39.3 ms in both configurations, with cached tokens unchanged at 448). There is no idle window immediately preceding it: control passes from **integrate** straight into **reason** with no tool call in between, so there is no time in which to prefill anything for it.

Second, **finalize** improves from 66.8 ms to 39.2 ms, a 1.71× reduction, with cached tokens rising from 304 to 496. It consumes the accumulated notes that **integrate** produced, and those notes are prefilled during the retrieval gap, so the terminal call arrives at an already-warm prefix.

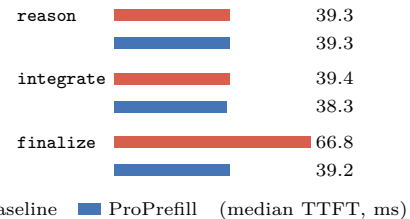


**Figure 3:** Per-component median TTFT for case study 1 (data of Table 1). Gains concentrate in the late stages, whose prompts are assembled from values produced earlier in the request.

Third, **integrate** improves only slightly, from 39.4 ms to 38.3 ms (1.03 $\times$ ), even though its cached tokens rise substantially, from 368 to 544. It too consumes a value produced by **reason** and prefilled in the gap, but the incremental prompt it adds per turn is small, so the absolute time removed is correspondingly small.



**Figure 4:** Case study 2: a single-agent ReAct loop with a data-dependent back-edge. The retrieval step opens an idle window of roughly 0.3 s per turn.



**Figure 5:** Case study 2 median TTFT per call site (data of Table 2).

The loop’s data dependence means the two configurations executed different numbers of **reason** and **integrate** calls (74 versus 84, and 58 versus 71); the comparison in Table 2 is therefore between per-call medians and not between matched call sequences. The **finalize** row, which is the row carrying the gain, is matched at 20 calls in both configurations.

## 5 Threats to Validity and Scope of the Claim

We state the boundaries of the evidence above explicitly.

**We report  $\Sigma$ TTFT, not end-to-end latency.** Summed TTFT is the quantity the mechanism acts on, and it is what we measure. We do not claim a proportional end-to-end win: the fraction of a request’s wall-clock time that  $\Sigma$ TTFT represents depends on decode lengths, tool latency, and retrieval cost, none of which ProPrefill changes.

**The Oracle column is an upper bound, and should be read as one.** Oracle is measured by replaying an identically-sized prompt and reading the second serve, so it reflects what the hardware does when every token is already cached. It upper-bounds what *any* KV-reuse system could achieve in this setting. The 72% figure in Table 1 is a fraction of that bound, not a fraction of total request latency.

**The hardware constrains the mechanism.** On an RTX 3090, a 4B model is large relative to the device. Prefill cannot be overlapped with decode without harming decode latency, so prefill–decode aggregation is disabled

in these runs. This cuts both ways: it removes a source of interference that would complicate the measurement, but it also means the spatial-idle path of Section 3 is not exercised here, and results on hardware where aggregation is viable may differ in both directions.

**The baseline is vLLM’s APC.** We build on vLLM, so the baseline adopts vLLM’s automatic prefix caching. We do not compare against the workflow-aware KV management systems discussed in Section 6; those systems govern a different stage of the KV lifecycle, and a fair comparison would require porting them onto the same runtime and hardware.

**Two topologies, one model, one GPU.** The evidence is two case studies at a single model size on a single device, with 20 requests each. They were chosen to be structurally different — a static fan-out with a supervisor branch, and a data-dependent loop with a self-referential carry — but they do not establish how the approach behaves under concurrency, at other model sizes, or on workloads whose idle windows are shorter than the prefill work available to fill them. The broader evaluation is ongoing.

**A configuration caveat, not a result caveat.** The measurements use compiler-derived speculative prefill with no probe decode and no deploy-time prewarm. The zero-gain rows for `route` and the four `analyze` stages are therefore properties of this configuration. We report them as measured and do not extrapolate what the probe mechanism would recover.

## 6 Related Work

### 6.1 Workflow-aware KV cache management

The closest body of work manages the KV cache of multi-agent workflows using knowledge of what will run next, and differs mainly in where that knowledge comes from.

**KVFlow** [7] abstracts the agent execution schedule as an Agent Step Graph and assigns each agent a steps-to-execution value estimating its temporal proximity to future activation; that value drives a node-level eviction policy and a fully overlapped prefetch that loads tensors from CPU to GPU in background threads for agents scheduled in the next step. Its target is explicitly the KV corresponding to agents’ *fixed* prompts. It reports up to 1.83 $\times$  speedup for single workflows with large prompts and up to 2.19 $\times$  with many concurrent workflows against SGLang with a hierarchical radix cache.

**PBKV** [14] targets *dynamic* workflows, where the sequence of invoked agents depends on the task. It predicts agent invocations several steps ahead by fusing guidance from historical workflows with the context of the

target workflow, scores cache entries by reuse potential, and applies the predictions conservatively in both eviction and prefetching. It reports up to 1.85 $\times$  over LRU on dynamic workflows and up to 1.26 $\times$  over KVFlow on the static workflow in its evaluation.

**CacheScout** [13] removes the offline dependency entirely: it learns agent execution transitions online, requiring no predefined workflow graph, no annotations, and no trace replay. It reports that predicting the next agent as the most frequent observed successor reaches 76–86% top-1 accuracy within 50 observed dispatches, and that recurring anchor content accounts for 53–62% of prompt tokens across its four evaluated workloads. It materializes predicted context by issuing a lightweight synthetic warmup request containing only the predicted agent’s reusable anchor.

**Pythia** [12] goes furthest along the content axis. It mines logged execution traces into probabilistic automata, identifies each request through three annotations attached at submission (`workflow_type_id`, `workflow_id`, `agent_id`), and prefills not only shared prefixes but argument content: upcoming prompts are assembled by injecting inputs and outputs recorded in the execution history, addressed through strict memory pointers into prior requests, and precomputed in idle GPU cycles. Its knowledge is the trace: a workflow or a path outside the history has nothing to instantiate.

**SAGA** [5] makes the workflow, rather than the individual call, the schedulable unit on a GPU cluster. Its agent execution graphs arrive from framework hints (orchestrator callbacks and message logs) or are inferred from request traces at 87% accuracy, with a fallback to request-level scheduling while a new agent type accumulates history; eviction is ranked by the predicted reuse of an agent’s successors, reported within 1.31 $\times$  of Bélády’s offline optimal on SWE-bench.

**Helium** [8] is the nearest neighbor on the compile-time axis. It treats an agentic workflow as a query plan: the developer composes the workflow in a dedicated DSL, LLM calls become plan operators, compilation identifies the static token sequences of each prompt, and the KV for those sequences is precomputed at the workflow’s first execution and reused afterward, with the plan’s leaves partitioned across workers to maximize reuse. It reports up to 1.56 $\times$  over KVFlow. The compiled knowledge stops at static prefixes: the plan is authored by the developer rather than recovered from a program, argument values are not speculated, and cache eviction remains the engine’s native LRU.

### 6.2 Static analysis of agent programs

AgentFlow [9] recovers agent dependencies from agent programs into a framework-agnostic Agent Dependency Graph, capturing component, control-flow and data-flow dependencies, and uses it for governance and security analyses such as Agent Bill of Materials generation and

prompt-to-tool risk detection; it is implemented for five frameworks and evaluated on a corpus of 5,399 real-world agent programs. It establishes that agent program structure is statically recoverable and that doing so is worth engineering, but it consumes that structure for security rather than for serving. ProPrefill shares the premise and applies it to the runtime’s KV schedule.

### 6.3 Serving substrates

ProPrefill builds on established serving machinery rather than replacing it. vLLM [6] supplies paged KV management and the automatic prefix caching used as our baseline; SGLang [15] established radix-structured prefix reuse for structured LM programs; Sarathi-Serve [1] contributes the chunked-prefill scheduling that lets us admit speculative prefill incrementally; and XGrammar [4] provides the constrained-decoding machinery whose grammar compilation we hoist into the idle window. The programming model is byLLM, the runtime of the meaning-typed abstraction introduced by MTP [3], which is what makes the prompt a compiler artifact in the first place.

## 7 Conclusion

Systems that serve agentic LLM programs have been trying to recover, at the serving layer, structure that the program already contains. When the LLM call is a language construct rather than a string handed across an API boundary, that structure is available to the compiler for free: the call topology, the provenance of every argument, and the invariant portion of every prompt fall out of a static pass with no annotation and no training. ProPrefill turns that knowledge into idle-time work, constructing the longest knowable prefix of a future call during the windows that tool calls and retrieval already open, and verifying it byte-for-byte against byLLM’s own rendering so that a wrong guess costs only a cache miss.

Two verification case studies on Qwen3-4B on a single RTX 3090 show the effect and its boundary. On a supervisor-plus-four-subagents workflow,  $\Sigma$ TTFT falls from 469.9 ms to 265.1 ms, capturing 72% of the headroom to a measured fully-cached Oracle, with the late, argument-heavy call sites improving by up to  $5.07\times$  and the call sites downstream of an unresolved branch improving not at all. On a ReAct loop, the terminal call site improves  $1.71\times$  while the call site with no preceding idle window is unchanged. In both cases the gain lands exactly where provenance says a value is available before its consumer runs, which is the strongest available evidence that the mechanism is doing what the design claims rather than something incidental. Extending the evaluation across more agent applications, model sizes, and concurrency levels is ongoing work.

The same artifacts reach past prefill. Provenance inverted is a liveness analysis: it names the call after which a binding’s KV has no remaining consumer and can be

freed deterministically instead of by recency. Invariant token counts fixed at compile time give every application a known warm working set and a known re-warm cost before it is deployed. Retention, eviction, and admission — decisions serving systems currently make by prediction — then become compiler outputs as well; that is the next stage of this co-design.

## References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. *arXiv preprint arXiv:2403.02310*, 2024.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [3] Jayanaka L. Dantanarayana, Yiping Kang, Kugesan Sivasothynathan, Christopher Clarke, Baichuan Li, Savini Kashmira, Krisztian Flautner, Lingjia Tang, and Jason Mars. MTP: A meaning-typed language abstraction for AI-integrated programming. *Proceedings of the ACM on Programming Languages*, 9 (OOPSLA2):Article 314, 2025. arXiv:2405.08965.
- [4] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for large language models. In *Proceedings of the 8th Conference on Machine Learning and Systems (MLSys)*, 2025. arXiv:2411.15100.
- [5] Dongxin Guo, Jikun Wu, and Siu Ming Yiu. SAGA: Workflow-atomic scheduling for AI agent inference on GPU clusters. *arXiv preprint arXiv:2605.00528*, 2026.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.
- [7] Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. *arXiv preprint arXiv:2507.07400*, 2025.



- [8] Noppanat Wadlom, Junyi Shen, and Yao Lu. Efficient LLM serving for agentic workflows: A data systems perspective. *arXiv preprint arXiv:2603.16104*, 2026.
- [9] Shenao Wang, Xinyi Hou, Yanjie Zhao, Xiao Cheng, and Haoyu Wang. AgentFlow: Building agent dependency graphs for static analysis of agent programs. *arXiv preprint arXiv:2607.01640*, 2026.
- [10] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- [12] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. Pythia: Exploiting workflow predictability for efficient agent-native LLM serving. *arXiv preprint arXiv:2604.25899*, 2026.
- [13] Rui Zhang, Chaeun Kim, Shaoting Feng, Kuntai Du, Yuhao Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. Learning agent execution for KV-Cache management in agentic serving. *arXiv preprint arXiv:2608.14624*, 2026.
- [14] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. Efficient serving for dynamic agent workflows with prediction-based KV-Cache management. *arXiv preprint arXiv:2605.06472*, 2026.
- [15] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.